Tài liệu ôn tập bảo vệ Nhóm 6

Verifier-Guided Repair: sửa chứng minh Lean 4 có kiểm chứng dưới ràng buộc chi phí và độ trễ

Môn INT4418 Dữ liệu lớn · Giảng viên: TS. Nguyễn Kiều Linh · Phiên bản nộp: tag v1.1

Phần A tổng hợp các câu hỏi khi chuẩn bị bảo vệ. Phần B đối chiếu sản phẩm với file hướng dẫn bài tập lớn. Phần C liệt kê các điểm còn thiếu và cách trả lời.

Phần A. Câu hỏi và trả lời

Các câu hỏi được gom theo chủ đề. Số liệu lấy từ báo cáo, run record và log trong results/raw. Số nào tính thêm từ log mà báo cáo chưa có thì được ghi rõ.

A1. Vai trò của Nhóm 6 trong chuỗi AI4Math

Bảng tổng quan của hướng dẫn ghi đầu ra kết nối của Nhóm 6 là Repaired Lean + error taxonomy, tức chứng minh Lean đã sửa kèm bảng phân loại lỗi. Nhóm 6 là khâu sửa lỗi: Nhóm 5 dịch đề toán sang Lean hàng loạt và nhiều kết quả bị Lean từ chối; Nhóm 6 biến các ứng viên hỏng đó thành chứng minh đã kiểm chứng và cho biết vì sao chúng hỏng.

Liên kết	Hướng dẫn quy định	Trạng thái
Nhóm 5 sang Nhóm 6	Có: dữ liệu của Nhóm 6 là candidate lỗi và compiler messages từ Nhóm 5	Có hợp đồng dữ liệu v0.1, cổng nhận repair_batch.py, file mẫu, hướng dẫn bàn giao. Chưa nhận dữ liệu thật
Nhóm 6 trả Nhóm 5	Không ghi	Cổng nhận trả feedback_group5.jsonl: trạng thái, nhóm lỗi, bản sửa
Nhóm 6 sang Nhóm 7	Không ghi	Dữ liệu repaired-v1 dùng được làm dữ liệu vết sửa; cần hỏi Nhóm 7
Nhóm 6 sang Nhóm 8	Không ghi trực tiếp; Nhóm 8 cần lời giải có kiểm chứng	Chứng minh trong repaired-v1 đều qua Lean và kiểm toán
Nhóm 3, 4 sang Nhóm 6	Không ghi	Chưa nối; hướng mở rộng: API tìm lemma để sửa lỗi sai tên ID_03
Nhóm 1, 2 sang Nhóm 6	Không ghi	Không dùng; nhóm tự lấy dữ liệu MIT và tự chia tập

Ví dụ một bài toán đi qua chuỗi: Nhóm 5 viết chứng minh bình phương mọi số thực không âm nhưng thiếu dòng import Mathlib; Lean báo unknown identifier. Nhóm 6 nhận file qua cổng nhận, xếp lỗi vào IMP_02, luật R0 thêm lại import, Lean chấp nhận, kiểm toán đạt, rồi ghi bản ghi repaired-v1 gồm mã trước và sau, nhãn lỗi và các lần thử.

> **Câu trả lời mẫu:** *Đầu vào chính thức của Nhóm 6 là ứng viên lỗi từ Nhóm 5, nhóm đã có hợp đồng dữ liệu và cổng nhận. Đầu ra là chứng minh đã sửa và kiểm chứng kèm nhãn lỗi*

theo định dạng repaired-v1. Các nhóm phía sau như Nhóm 7 cần dữ liệu vết hay Nhóm 8 cần lời giải có kiểm chứng đều có thể dùng đầu ra này. Hướng nối tiếp theo là gọi API tìm lemma của Nhóm 3 hoặc Nhóm 4 để sửa lỗi sai tên.

A2. Vai trò của Tâm và kiến thức cần nắm

Tâm là trưởng nhóm, vai trò chính Evaluation và Cost Analysis, dự phòng Queue và Worker System. Khi bảo vệ, Tâm trình bày slide 1, 2, 3, 8, 10, 14, 15, khoảng 8 phút, và trả lời câu hỏi về câu hỏi nghiên cứu, đánh giá, chi phí.

Theo lịch sử git, Tâm commit 58 lần vào src, scripts, tests và tạo 33 trên 40 file mã hiện tại, có Claude hỗ trợ, đã khai báo trong docs/ai_use_log.jsonl. Mã thuộc vai trò cần nắm chắc:

File	Vai trò
scripts/run_experiment.py	Chạy một cấu hình, tính chỉ số, ghi run record
src/cost_guard.py	Kiểm tra chi phí xấu nhất trước mỗi lần gọi LLM, trần 8 USD mỗi tháng và 2 USD mỗi lần chạy, sổ chi tiêu, bộ đệm
scripts/estimate_cost.py	Ước tính chi phí trước khi chạy
src/provenance.py	Ghi run record theo Phụ lục B
scripts/generate_figures.py	Sinh hình và bảng từ kết quả thô
scripts/analyze_failures.py	Lý do dừng, độ chính xác từng luật
scripts/parser_ablation.py	Ablation bộ phân loại, phát lại log

Câu trả lời mẫu: *Em là người viết và tích hợp phần lớn mã, có Claude hỗ trợ. Việc dùng AI được khai báo trong nhật ký, mỗi dòng ghi nội dung và cách kiểm chứng. Mọi đoạn mã đều qua 121 bài kiểm thử và Lean thật, mọi số liệu truy được về run record. Em chịu trách nhiệm về thiết kế thực nghiệm, độ đo và kiểm soát chi phí.*

A3. Thiết kế thực nghiệm

Hold-out: 284 mẫu ở mức 1K, 2.907 mẫu ở mức 10K

Nhóm tạo ba bộ dữ liệu 100, 1.000 và 10.000 mẫu lỗi. Mỗi bộ chia khoảng 70% train, được xem khi thiết kế luật và prompt, và khoảng 30% hold-out, cất riêng để chấm điểm. Bộ 1K có 716 mẫu train và 284 mẫu hold-out; bộ 10K có 2.907 mẫu hold-out. Con số không tròn vì chia theo định lý gốc. RQ1, RQ2 đo trên 284 mẫu; RQ3 đo tốc độ trên toàn bộ mỗi mức.

Chia theo định lý gốc

Từ mỗi định lý đúng, nhóm tạo tối đa 5 bản lỗi. Nếu chia ngẫu nhiên từng mẫu, hai bản lỗi của cùng định lý có thể nằm ở hai phía và hệ thống được thấy trước đáp án. Chia theo định lý gốc giữ mọi bản lỗi của một định lý ở cùng một phía. Bộ 1K có 649 định lý train, 260 định lý hold-out, 0 định lý ở cả hai phía. Hàm split_for băm tên định lý nên chạy lại luôn ra cùng cách chia.

Nhiệt độ 0

Khi sinh từng token, LLM có một bảng xác suất cho token tiếp theo. Nhiệt độ 0 luôn chọn token có xác suất cao nhất, nên cùng đầu vào cho cùng đầu ra. Nhiệt độ cao hơn chọn ngẫu nhiên theo xác suất. Nhóm đặt nhiệt độ 0 để tái lập được, so sánh công bằng và dùng được bộ đệm. Đánh đổi: ba lần lặp của cấu hình LLM cho cùng kết quả nên không đo được dao động của API; ở mỗi lần thử, vòng lặp phải gửi thông báo lỗi mới để mô hình có thông tin khác.

Seed là gì, vì sao 42, dùng ở đâu

Máy tính sinh dãy số giả ngẫu nhiên từ một con số khởi đầu gọi là seed; cùng seed thì cùng dãy. 42 là quy ước phổ biến, không có lý do kỹ thuật. Độ lớn của seed không có ý nghĩa: seed 42 và 43 cho hai cách xáo hoàn toàn khác nhau, seed lớn không ngẫu nhiên hơn. Seed 42 được dùng ở bốn chỗ:

Chỗ dùng	Code	Tác dụng
Chọn định lý gốc	build_seeds.py: random.Random(args.seed).shuffle(rows)	Luôn chọn cùng các chứng minh từ 29.750 bài
Chọn cách gây lỗi	generate_error_dataset.py: random.Random(f"{seed}:{seed_id}")	Mỗi định lý có thứ tự 9 cách gây lỗi riêng, không phụ thuộc thứ tự xử lý
Xếp thứ tự, tạo bộ 100, 1K, 10K	generate_error_dataset.py: shuffle rồi records[:1000]	Bộ nhỏ nằm trong bộ lớn; mã E000880 luôn là cùng một mẫu
Gọi Gemini	llm_client.py: seed=self.seed	Ổn định câu trả lời cùng với nhiệt độ 0

Việc chia train và hold-out không dùng seed mà dùng phép băm tên định lý. Hạn chế: mới sinh dữ liệu với một seed; hướng mở rộng là chạy nhiều seed để đo dao động do chọn dữ liệu.

Ma trận thí nghiệm: 48 lần chạy trên Kaggle

Ma trận	Cấu hình	Số lần chạy
Chất lượng, 284 mẫu hold-out	Không sửa k0; chỉ luật k3; baseline k1; LLM có chẩn đoán k1 và k3; kết hợp k1 và k3; LLM có chẩn đoán dùng Gemini 3.6 Flash k1; kết hợp k1 trên hold-out 10K	21
Mở rộng, chỉ dùng luật	3 mức 100, 1K, 10K nhân 3 mức 1, 2, 4 worker nhân 3 lần lặp	27

Lặp 3 lần khác k bằng 0, 1, 3

	k bằng 0, 1, 3	Lặp 3 lần
Là gì	Số lần được thử sửa tối đa cho mỗi mẫu	Chạy lại toàn bộ một thí nghiệm
Để làm gì	Đo thử nhiều lần có sửa được nhiều hơn không	Đo độ ổn định; báo trung vị và khoảng nhỏ nhất đến lớn nhất

Vì sao không có k bằng 2

Đề bài quy định retry budget 0, 1, 3. Vì nhiệt độ 0 và vòng lặp tất định, kết quả k bằng 2 tính chính xác từ log của k bằng 3 bằng cách dừng sau lần thử thứ hai. Kiểm chứng chéo: cách tính cho k bằng 1 ra đúng 53,9% và 1,13 USD, khớp lần chạy thật. Số liệu tính từ log, chưa có trong báo cáo:

k	Thành công	Thêm so với mức trước	Chi phí cho 1.000 mẫu
1	53,9% (153/284)		1,13 USD
2	62,0% (176/284)	+23 mẫu	1,84 USD
3	65,1% (185/284)	+9 mẫu	2,45 USD

Mỗi lần thử thêm mang lại ít hơn: lần thử thứ 3 thêm 3,1 điểm, chi phí cho mỗi mẫu sửa thêm gấp khoảng 2 lần lần thử thứ 2.

Baseline của nhóm

Baseline là thử lại nguyên prompt đúng một lần, không phân loại lỗi, không gửi thông báo lỗi, tên chiến lược blind. Đây là baseline bắt buộc ở mục 7 của phần Nhóm 6. Chạy cùng 284 mẫu, cùng mô hình, nhiệt độ 0, seed 42, timeout 60 giây. Kết quả 31,7% với 2,05 USD cho 1.000 mẫu, so với 53,9% và 1,13 USD khi có thông báo lỗi.

Ablation

Ablation là bỏ hoặc thay đổi từng thành phần để đo đóng góp của nó. Đề bài tuần 8 yêu cầu ablation parser, router, context.

So sánh	Bỏ đi	Kết quả
Baseline so với LLM có chẩn đoán, k1	Thông báo lỗi, tức context	+22,2 điểm, đây là RQ1
Chỉ luật so với LLM	LLM	Luật một mình đạt 23,9%
LLM so với kết hợp	Luật và router	Router hiện làm kết quả kém đi
k1 so với k3	Các lần thử lại	+11,2 điểm
Biên dịch so với đã kiểm toán	Bước kiểm toán	Thổi phồng 9,9 điểm nếu bỏ kiểm toán
Ablation bộ phân loại	Lọc lỗi hệ quả, suy luận thiếu import	Độ khớp nhãn 72,3% lên 96,3%, token giảm 31%

LLM k1 so với Gemini 3.6 Flash k1

	Gemini 3.5 Flash-Lite, k1	Gemini 3.6 Flash, k1
Giá cho 1 triệu token vào và ra	0,30 và 2,50 USD	0,75 và 3,75 USD
Thành công đã kiểm toán	53,9%	36,6%
Biên dịch	59,2%	44,0%
Chi phí cho 1.000 mẫu	1,13 USD	1,89 USD
P95 thời gian phục vụ	16,4 giây	18,9 giây

Chỉ đổi mô hình, mọi thứ khác giữ nguyên. Mô hình lớn hơn vừa đắt hơn vừa kém hơn, gây nhiều lỗi biên dịch hơn và hay đổi đề hơn. Cả hai chạy ở mức suy nghĩ tối thiểu nên nhóm không kết luận mô hình lớn kém hơn.

A4. Độ đo và chi phí

Token vào và token ra

Token vào là phần gửi cho mô hình: hướng dẫn, mã lỗi, thông báo lỗi. Token ra là bản sửa mô hình viết. Với Gemini 3.5 Flash-Lite, token ra đắt gấp 8,3 lần token vào. Ví dụ LLM k3: 357.188 token vào tốn 0,107 USD, 235.482 token ra tốn 0,589 USD, tổng 0,696 USD; chia 284 mẫu nhân 1.000 ra 2,45 USD cho 1.000 mẫu. Token ra chiếm gần 85% chi phí.

Gửi thông báo lỗi làm prompt dài hơn 68%, nhưng mô hình chỉ sửa đúng chỗ, trung bình 389 token ra thay vì 783, nên tổng chi phí giảm 45%.

P50, P95 và các phân vị khác

P là phân vị. P95 bằng X giây nghĩa là 95% số mẫu xử lý xong trong X giây. Ví dụ thời gian xử lý từng mẫu của LLM k3:

P0	P25	P50	P75	P90	P95	P99	P100	Trung bình
0,0 s	0,8 s	2,6 s	6,5 s	11,8 s	16,4 s	38,4 s	57,5 s	5,2 s

Trung bình 5,2 giây che mất cả số đông nhanh lẫn 14 mẫu chậm hơn 16,4 giây, nên đề bài bắt buộc báo cáo P95. P0 là nhỏ nhất, P100 là lớn nhất; P25 và P75 là tứ phân vị; P99 và P99.9 dùng cho hệ thống lớn.

Tăng compile rate có đủ khi chi phí token tăng mạnh

Không đủ, vì hai lý do. Thứ nhất, compile rate bị thổi phồng: LLM k3 biên dịch 75,0% nhưng thành công thật 65,1%, và khoảng chênh tăng khi thử nhiều lần. Thứ hai, phải xét chi phí biên. Số liệu tính từ bảng ablation:

Cấu hình	Thành công	Chi phí cho 1.000 mẫu	Chi phí trung bình mỗi mẫu sửa được
Chỉ luật	23,9%	0 USD	0 USD
LLM k1	53,9%	1,13 USD	0,0021 USD
Kết hợp k3	59,5%	1,81 USD	0,0030 USD
LLM k3	65,1%	2,45 USD	0,0038 USD
Gemini 3.6 Flash k1	36,6%	1,89 USD	0,0052 USD
Baseline k1	31,7%	2,05 USD	0,0065 USD

Từ k2 lên k3, chi phí cho mỗi mẫu sửa thêm khoảng 0,019 USD, gấp khoảng 9 lần mức trung bình của k1. Độ trễ P95 cũng tăng từ 16,4 lên 19,3 giây. Cách quyết định: đặt trần ngân sách rồi chọn cấu hình trên biên Pareto; tăng thêm khi chi phí biên còn thấp hơn giá trị một mẫu sửa được; phân tầng luật chính xác trước, LLM rẻ sau.

A5. Khái niệm hệ thống

Lean chấp nhận, ai chỉ nhận bản sửa

Lean chấp nhận nghĩa là Lean kiểm tra mọi bước và không báo lỗi. Hệ thống của Nhóm 6, ở bước cuối vòng lặp, chỉ tính một bản sửa là thành công khi qua hai cửa: Lean chấp nhận và bộ kiểm toán đạt. Trượt một cửa thì thử lại nếu còn lượt.

Trình kiểm chứng và kiểm toán

Trình kiểm chứng là Lean. Kiểm toán là bước nhóm tự viết trong src/semantic_auditor.py, vì Lean chỉ kiểm tra chứng minh đúng với phát biểu được viết ra, không biết phát biểu đó có đúng đề bài ban đầu không. Kiểm toán yêu cầu: Lean chấp nhận; không sorry; không axiom, admit, native_decide; phát biểu định lý giữ nguyên sau khi bỏ khác biệt khoảng trắng. Ví dụ gian lận Lean vẫn chấp nhận: đổi $0 < x * x$ thành $0 \leq x * x$.

REPL

REPL là chế độ chạy liên tục: nhận lệnh, thực thi, trả kết quả, chờ lệnh tiếp. Chạy nguội phải nạp Mathlib cho mỗi tệp, mất 63 đến 107 giây. Lean REPL nạp Mathlib một lần, sau đó mỗi định lý chỉ P50 3 ms, P95 23 ms, khớp 74 trên 74 lần với chạy nguội. Nhóm bổ sung khởi động lại khi REPL chết và tắt cả nhóm tiến trình để không rò tiến trình.

Gửi thông báo lỗi

Prompt baseline chỉ gửi mã và yêu cầu sửa. Prompt có chẩn đoán gửi mã có đánh số dòng, kèm tối đa 5 thông báo lỗi của Lean với vị trí và nhãn nhóm lỗi, yêu cầu sửa nguyên nhân của lỗi đầu tiên. Mẫu nằm ở configs/prompts/repair_v1.yaml.

Lập lịch thể hiện ở đâu

- Hàng đợi FIFO trong VerifierPool chia mẫu cho các worker, mỗi worker giữ một Lean REPL.
- Đo riêng thời gian chờ trong hàng và thời gian xử lý.
- Chọn số worker: từ 2 lên 4 worker chỉ thêm 7 đến 11% thông lượng, P95 tăng gấp đôi vì 4 worker tranh 4 nhân CPU.
- Giới hạn mỗi mẫu 300 giây và k lần thử để một mẫu khó không chiếm worker mãi.
- Hướng phát triển: hàng đợi ưu tiên; Dũng đã có nguyên mẫu Redis.

Kiến trúc

Tầng	Thành phần	File
Dữ liệu	Định lý gốc, 9 cách gây lỗi, nhãn từ Lean, chia theo định lý, 3 mức	build_seeds.py, generate_error_dataset.py, manifest.json
Sửa lỗi	Phân loại, định tuyến, luật hoặc LLM qua kiểm soát chi phí, Lean REPL, kiểm toán	error_parser.py, repair_pipeline.py, rule_repair.py, llm_repair.py, cost_guard.py, lean_repl.py, semantic_auditor.py
Lập lịch	Hàng đợi FIFO, W worker, khởi động lại khi REPL chết	verifier_pool.py
Ghi vết	Run record, log từng mẫu, sổ chi tiêu, hình sinh từ kết quả thô	provenance.py, generate_figures.py
Tích hợp	Cổng Nhóm 5, repaired-v1, Docker, notebook Kaggle, nguyên mẫu Redis	repair_batch.py, export_repaired.py, Dockerfile

Vòng lặp dừng khi thành công, hết k lần, quá 300 giây, hoặc bản sửa trùng bản đã thử. Thời gian chạy xấp xỉ T bằng T0 cộng N nhân k nhân (v cộng p nhân l) chia W, với T0 thời gian nạp Mathlib, N số mẫu, k số lần thử trung bình, v thời gian kiểm chứng, p tỷ lệ lần thử gọi LLM, l độ trễ LLM, W số worker.

A6. Bộ phân loại và bộ định tuyến

Bảy nhóm lỗi

Mã	Loại lỗi	Hướng sửa	Số mẫu
SYN_01	Cú pháp	Luật	4.590
ID_03	Sai tên, lemma không tồn tại	Thử luật rồi LLM	3.978
SEM_06	Chứng minh để trống bằng sorry	Kiểm toán, LLM	2.292
IMP_02	Thiếu import	Luật	2.269
TAC_07	Bước chứng minh thất bại	LLM	1.827
TMO_05	Vượt giới hạn tính toán	Chỉnh cấu hình	1.707
TYP_04	Sai kiểu	LLM	338
UNK_99	Không xếp được	LLM	30

Độ tin cậy, tính từ dữ liệu

Trên 15.921 lỗi tổng hợp, độ khớp với loại lỗi cố ý gây ra là 94,4%. Theo từng nhóm, số tính thêm chưa có trong báo cáo: IMP_02, SEM_06, TMO_05, SYN_01 khoảng 100%; TAC_07 99,2%; ID_03 97,3%; TYP_04 chỉ 29,4% vì đổi kiểu số thường làm Lean báo bước chứng minh thất bại chứ không báo sai kiểu, nên nhãn của bộ phân loại vẫn đúng

với lỗi thật. Bỏ TYP_04, độ khớp là 99,4%. Nhãn tái lập 17.031 trên 17.031. Hạn chế: lỗi tự nhiên không có đáp án để so; chỉ xét một lỗi gốc; phân loại đúng chưa chắc sửa được.

Khả năng mở rộng: so khớp từ khoá mất vài micro giây mỗi mẫu, không phải nút thắt. Thêm nhóm lỗi chỉ cần một dòng trong _RULES kèm test, như đã làm với TAC_07. Rủi ro là Lean đổi cách diễn đạt; cần chạy lại test và theo dõi tỷ lệ UNK_99.

Bộ định tuyến hoạt động thế nào

- Bộ phân loại so khớp từ khoá trong thông báo lỗi gốc, gán nhóm lỗi và hướng sửa.
- Chỉ chiến lược hybrid thực sự dùng định tuyến: IMP_02, SYN_01, ID_03, TMO_05 thử luật trước; TYP_04, TAC_07, SEM_06, UNK_99 gửi thẳng LLM. rule_only luôn dùng luật; blind và llm_only luôn dùng LLM.
- Chọn luật theo nhóm: IMP_02 dùng R0; TMO_05 dùng R7; SYN_01 dùng R6, R1, R2; ID_03 dùng R5, R4.
- Luật không thay đổi mã hoặc đã thử thì hybrid chuyển sang LLM, rule_only dừng.

Xác định lỗi nào dùng chiến lược nào

Ban đầu nhóm gán theo suy luận về bản chất lỗi. Thực nghiệm theo nhóm lỗi cho thấy chỉ IMP_02 và TMO_05 nên dùng luật:

Nhóm lỗi	Chỉ luật	LLM có chẩn đoán k1	Nên dùng
IMP_02	100%	34%	Luật
TMO_05	93%	69%	Luật
SYN_01	12%	85%	LLM
ID_03	0%	60%	LLM
TAC_07	0%	17%	LLM
SEM_06	0%	24%	LLM

Ước tính ghép từ log, chưa có trong báo cáo: dùng luật cho IMP_02, TMO_05 và LLM cho phần còn lại, một lần thử, đạt 63,7% với 0,97 USD cho 1.000 mẫu, so với hybrid hiện tại 30,3% và LLM k1 53,9%. Con số lạc quan vì cách định tuyến được chọn từ chính hold-out; cách đúng là đo trên train rồi chấm trên hold-out.

A7. Đổi mô hình LLM

LLM giao tiếp qua đúng một hàm complete. Đổi sang mô hình Gemini khác chỉ cần tham số --model và khai báo giá; nhà cung cấp khác cần một client khoảng 50 dòng. Verifier, kiểm toán và hold-out giữ nguyên nên so sánh được ngay.

DeepSeek-Prover-V2 7B chạy miễn phí trên GPU Kaggle, không cần API key, nhưng ước tính 3 đến 5 giờ GPU, được huấn luyện trên Mathlib cũ hơn bản 4.35 nên dễ gặp lỗi tên lemma đã đổi. Nhóm đưa vào hướng phát triển.

> **Câu trả lời mẫu:** *Nhóm chọn Gemini 3.5 Flash-Lite vì rẻ, nhanh và tái lập được. Câu hỏi nghiên cứu là về kiến trúc: thông báo lỗi giúp tăng từ 31,7% lên 53,9% là so sánh trong cùng một mô hình. Mô hình lớn hơn Gemini 3.6 Flash chỉ đạt 36,6%. Hướng hợp lý là phân tầng theo chi phí: luật chính xác trước, mô hình rẻ sau, mô hình đắt chỉ cho lỗi khó.*

A8. Demo

Demo phát lại log thực nghiệm nên không cần Lean, mạng hay chi phí. Sau khi git clone, chỉ cần Python 3.10 trở lên, không cài thư viện; ba mẫu demo nằm trong data_sample/ demo_samples.jsonl.

```
HYB=results/raw/logs/exp-hybrid-b3-w4-20261017102347-852e82.jsonl.gz
LLM=results/raw/logs/exp-llm_only-b3-w4-20261017101714-3f7a7e.jsonl.gz
python3 scripts/demo_repair.py --replay $HYB --error-id E000880   # luật R0, 0
token, 73 ms
python3 scripts/demo_repair.py --replay $LLM --error-id E000003   # lần 1 dùng
sorry bị chặn, lần 3 đúng, 4.583 token
python3 scripts/demo_repair.py --replay $LLM --error-id E000197   # sửa đúng ở lần
2, 779 token
```

A9. Nguồn khởi đầu [2], [3], [4], [10]

Bốn nguồn là hai công trình: ProofNet, gồm repo [2] và bài báo [10], và LeanDojo, gồm tài liệu [3] và website [4]. Cả hai được trích dẫn trong báo cáo, LeanDojo là [1], ProofNet là [3].

Nguồn	Đóng góp cho dự án	Vì sao không dùng trực tiếp
ProofNet	Định hướng giao diện với Nhóm 5: hợp đồng dữ liệu có trường source_dataset ProofNet; bối cảnh ở mục 2 báo cáo	Bản gốc Lean 3, chỉ 371 bài, không có chứng minh hoàn chỉnh; ứng viên sinh từ ProofNet là việc của Nhóm 5. Nhóm dùng gần 30.000 chứng minh Lean 4 của Lean Workbook, giấy phép MIT
LeanDojo	Ý tưởng tương tác với Lean bằng chương trình; tìm kiếm lemma là hướng mở rộng cho ID_03	Nhóm cần kiểm chứng hàng loạt ở tốc độ cao nên dùng Lean REPL, 3 ms mỗi định lý. Đây là lý do theo yêu cầu hệ thống, không phải kết quả đo so sánh hai công cụ

A10. Yêu cầu dữ liệu

Yêu cầu	Đáp ứng
Không đưa dữ liệu lớn vào Git	data/ và lean_ws/.lake bị .gitignore loại; repo 16,7 MB; file dữ liệu lớn nhất trong git là log nén 1,2 MB
Chỉ lưu mẫu hợp pháp	data_sample/ dưới 400 KB mỗi file, sinh từ nguồn giấy phép MIT
Manifest và checksum	manifest.json ghi phiên bản v1.0, nguồn, giấy phép, SHA-256; 11 trên 11 checksum khớp file trên máy
Script tải hoặc tạo dữ liệu	download_data.sh kiểm tra SHA-256; build_seeds.py, generate_error_dataset.py tạo lại toàn bộ
Split cố định, có phiên bản	split_for băm tên định lý; run record ghi data_version, ví dụ errors_1k.jsonl@63704690f69c

Vấn đề còn lại: docs/references có ba sách PDF trong repo công khai, trong đó Data Mining Concepts and Techniques là sách thương mại. Không phải dữ liệu, nhưng nên gỡ khỏi git và thay bằng link chính thức.

Phần B. Đối chiếu với hướng dẫn bài tập lớn

Theo thứ tự của HuongDan_Bai_Tap_Lon_BigData_INT4418.pdf: phần chung, phần Nhóm 6, các phụ lục mẫu A, B, C. Phần của Nhóm 1 đến Nhóm 5 là đề bài của nhóm khác, chỉ đối chiếu ở các điểm có liên kết với Nhóm 6.

B1. Cơ cấu nhóm và mối liên hệ giữa các đề tài

Yêu cầu	Trạng thái	Minh chứng
Đầu ra kết nối của Nhóm 6: Repaired Lean + error taxonomy	Đạt	docs/output_contract_group6.md, data/repaired, data_sample/repaired_sample_50.jsonl, docs/error_taxonomy_v0.2.md
Thống nhất interface ở tuần 2	Đạt	docs/data_contract_group5_group6.md, mốc tuần 2
Có snapshot hoặc mock input, không phụ thuộc tiến độ nhóm khác	Đạt	Dataset tự sinh 17.031 mẫu; data_sample/group5_input_sample.jsonl

Nhóm khác	Liên hệ với Nhóm 6	Trạng thái
1 Math Data Lake	Snapshot, data contract	Không dùng; nhóm tự lấy nguồn MIT
2 Data Quality	Clean splits	Không dùng; nhóm tự chia theo định lý và kiểm tra trùng
3, 4 Lean Retrieval	API tìm lemma	Chưa nối; hướng mở rộng cho ID_03
5 Autoformalization	Candidate Lean + error logs, đầu vào chính thức	Hợp đồng và cổng nhận sẵn sàng; chưa có dữ liệu thật

B2. Yêu cầu chung bắt buộc

Yêu cầu	Trạng thái	Minh chứng
Câu hỏi nghiên cứu chỉ ra nút thắt Big Data	Đạt	Báo cáo mục 1: quy mô, lập lịch, chi phí; kiểm chứng nguội 63 đến 107 giây
Baseline đơn giản chạy end-to-end trước phương pháp nâng cao	Đạt	Baseline blind và chỉ luật; bản giả lập tuần 3, bản thật trên Lean ở tuần 7
Ít nhất ba mức quy mô, cách nhau gần một bậc độ lớn	Đạt	100, 1K, 10K; 1, 2, 4 worker
Báo cáo đồng thời chất lượng và chi phí	Đạt	Bảng ablation, biểu đồ Pareto
Biểu đồ sinh từ raw bằng script; ghi phần cứng, phần mềm, seed, timeout, số lần lặp	Đạt	generate_figures.py; run record; báo cáo mục 5
Error analysis, ablation, limitation	Đạt	Báo cáo mục 7.1 đến 7.4
Code, data sample, data/model card, README cho phép chạy lại	Đạt	README, data card, system card; không huấn luyện mô hình nên system card thay cho model card
Vai trò chính và dự phòng; hiểu toàn bộ pipeline	Đạt	README mục Thành viên, contributions.md, slides/defense_qa.md
Khai báo sử dụng AI; nhóm kiểm chứng	Đạt	docs/ai_use_log.jsonl, docs/ai_use_statement.md, 121 test

B3. Tiến độ chung 10 tuần

Tuần	Minh chứng yêu cầu	Trạng thái	Minh chứng của nhóm
1	Đề cương 1 trang và board nhiệm vụ	Đạt	docs/de_cuong_nhom_6_tuan_1.md, docs/task_checklist_ai_vs_human.md
2	Data sample và data card v0.1	Đạt	data_sample/, docs/data_card_lean_errors.md, hợp đồng với Nhóm 5
3	Code, raw metric, demo nội bộ của baseline	Một phần	Run record baseline tuần 3 là bản giả lập; baseline thật chạy tuần 7, 31,7%
4	Checkpoint report, kiểm thử	Đạt	tests/, luật R1 đến R4 và benchmark của Trung
5, 6	Pipeline v0.2, test, environment lock	Đạt	VerifierPool, khôi phục khi REPL chết, requirements-lock.txt
7	Raw CSV hoặc JSON và cấu hình từng run, lặp 3 lần	Đạt	48 lần chạy trên Kaggle
8	Bảng ablation, taxonomy lỗi, limitations	Đạt	table_ablation.md, table_parser_ablation.csv, báo cáo mục 7
9	Release candidate, report, demo script	Đạt	Tag v1.0, báo cáo 20 trang, slides/demo_script.md
10	Final release, contribution log	Đạt	Tag v1.1, contributions.md; còn buổi bảo vệ

B4. Cấu trúc sản phẩm bàn giao

Yêu cầu	Trạng thái	Minh chứng
README: mục tiêu, kiến trúc, cài đặt, dữ liệu, lệnh chạy, tái tạo kết quả	Đạt	README.md, có mục Tóm tắt dễ hiểu
src, tests, configs	Đạt	121 test qua
data_sample hợp pháp, scripts tải hoặc tạo dữ liệu	Đạt	manifest.json, download_data.sh
results/raw và results/figures sinh bằng script	Đạt	67 run record
docs: data card, model/system card, AI-use, privacy/license	Đạt	Đủ bốn loại tài liệu
report.pdf/docx	Một phần	Mục 4 ghi 8 đến 12 trang, Phụ lục C ghi 12 đến 20 trang; báo cáo hiện 20 trang
slides và kịch bản trình diễn	Đạt	15 slide và 2 phụ lục, demo_script.md, kịch bản thuyết trình
contributions.md: vai trò, issue, pull request, phần bảo vệ	Đạt	Bảng PR #1 đến #14, phân công bảo vệ

B5. Rubric đánh giá

Thành phần	Tỷ trọng	Minh chứng
Câu hỏi và vai trò Big Data	10%	3 RQ, nút thắt quy mô, lập lịch, chi phí; giả thuyết đo được
Dữ liệu và provenance	15%	Nguồn MIT, phiên bản v1.0, checksum, schema, độ khớp nhãn 94,4%
Kiến trúc và triển khai	20%	Pipeline ổn định, 121 test, run record, log, hợp đồng vào và ra
Baseline	10%	Baseline bắt buộc, cùng điều kiện, tái lập
Scalability experiments	20%	3 mức quy mô, 3 mức worker, 3 lần lặp, kết quả thô
Đánh giá và phân tích	10%	Ablation, phân tích lỗi, Pareto chi phí và chất lượng
Bàn giao và tái lập	10%	README, Docker, requirements-lock, data_sample, demo chạy sau git clone
Trình bày và đóng góp cá nhân	5%	Slide, kịch bản, bộ 28 câu hỏi; phụ thuộc buổi bảo vệ

B6. Phần Nhóm 6: Verifier-Guided Repair

Mục	Yêu cầu	Trạng thái	Minh chứng hoặc ghi chú
1	Câu hỏi: thông báo lỗi có cấu trúc so với retry	Đạt	RQ1: 31,7% lên 53,9%, chi phí giảm 45%
1	Câu hỏi: chính sách retry và worker dưới ngân sách	Đạt	RQ2 chiến lược và k; RQ3 số worker
2	Candidate lỗi từ Nhóm 5	Một phần	Hợp đồng, cổng nhận, file mẫu; chưa có dữ liệu thật
2	Bộ lỗi tổng hợp có nhãn 6 loại	Đạt	15.921 lỗi tổng hợp, 1.110 lỗi tự nhiên, thêm TAC_07
2	Tập hold-out không dùng để thiết kế luật	Đạt	Chia theo định lý, 0 trùng
2	Yêu cầu dữ liệu	Đạt	Xem A10
3	Verifier, parser, taxonomy, router, rule/ LLM repair, bounded loop, queue và result store	Đạt	Đủ 7 thành phần; thêm kiểm toán và kiểm soát chi phí
4	Lean 4, Python, queue, LLM API hoặc mô hình mở, Docker	Đạt	Gemini API; Docker đã build thử; chưa thử mô hình mở
4	Prometheus hoặc logging, không bắt buộc	Đạt	Logging qua run record và log
5	Sáu vai trò, mỗi vai trò có người dự phòng	Đạt	README; phân bổ mã nguồn lệch về Tâm, đã ghi trong contributions.md
6	Nhiệm vụ 10 tuần	Một phần	Đủ minh chứng; baseline tuần 3 là bản giả lập
7	Baseline retry nguyên prompt một lần, cùng điều kiện	Đạt	Chiến lược blind k1
8	1K đến 10K; 1, 2, 4 worker; budget 0, 1, 3; rule, LLM, hybrid; có và không có thông báo lỗi	Đạt	Đủ năm trục
8	FIFO so với priority nếu đủ thời gian	Chưa	Mới có nguyên mẫu Redis chạy giả lập
8	Ghi đủ thông tin mỗi lần chạy, lặp 3 lần, median và khoảng	Đạt	Run record theo Phụ lục B
9	Chỉ số chất lượng và hiệu năng, chọn 1 đến 2 chỉ số chính	Đạt	Thành công đã kiểm toán; thông lượng
10	8 sản phẩm bàn giao	Đạt	Error set, parser, router, worker queue, log, đường cong chi phí, test, README và demo
11	6 tiêu chí nghiệm thu tối thiểu	Đạt	Đạt 6 trên 6
12	Hướng phát triển thành bài báo	Một phần	Có khoảng trống, ablation, threat to validity; chưa có baseline prover chuyên Lean

B7. Các phụ lục mẫu

Phụ lục	Yêu cầu	Trạng thái	Minh chứng
A	Đề cương 1 trang tuần 1	Đạt	docs/ de_cuong_nhom_6_tuan_1.md
B	Mẫu ghi một lần chạy: run_id, timestamp, commit, data version, machine, software, parameters, seed, timeout, metrics, status, log	Đạt	src/provenance.py; mọi file results/ raw/run_*.json
C	Cấu trúc báo cáo 12 đến 20 trang, 8 mục và phụ lục	Đạt	Báo cáo 20 trang, đủ 8 mục, phụ lục AI-use, đóng góp, lệnh tái lập; 8 trích dẫn trong khoảng 3 đến 8
D	Danh mục nguồn chung	Đạt	Xem A9

Phần C. Điểm còn thiếu và cách trả lời

Điểm	Cách trả lời hoặc xử lý
Chưa có dữ liệu thật của Nhóm 5	Hướng dẫn cho phép mock input; hợp đồng và cổng nhận đã sẵn sàng, chạy được ngay khi có file
FIFO so với priority chưa đo trên Lean thật	Đề bài ghi nếu đủ thời gian; nguyên mẫu Redis có, báo cáo ghi ở hướng phát triển
Baseline tuần 3 là bản giả lập	Baseline thật trên Lean chạy ở tuần 7, cùng điều kiện với phương pháp chính
Chưa có baseline prover chuyên Lean	Đã ước tính DeepSeek-Prover; đưa vào hướng phát triển
Phân bổ mã nguồn lệch về Tâm	Ghi trung thực trong contributions.md; mỗi thành viên bảo vệ mảng của mình
Độ dài báo cáo 8 đến 12 hay 12 đến 20 trang	Hỏi giảng viên; báo cáo hiện 20 trang theo Phụ lục C
Sách có bản quyền trong docs/ references	Nên gỡ khỏi git, thay bằng link chính thức
Số liệu tính thêm chưa có trong báo cáo	k2, định tuyến theo số liệu, độ khớp theo nhóm lỗi: nói rõ là tính từ log